CS 412 / 512
Introduction to Computer Graphics

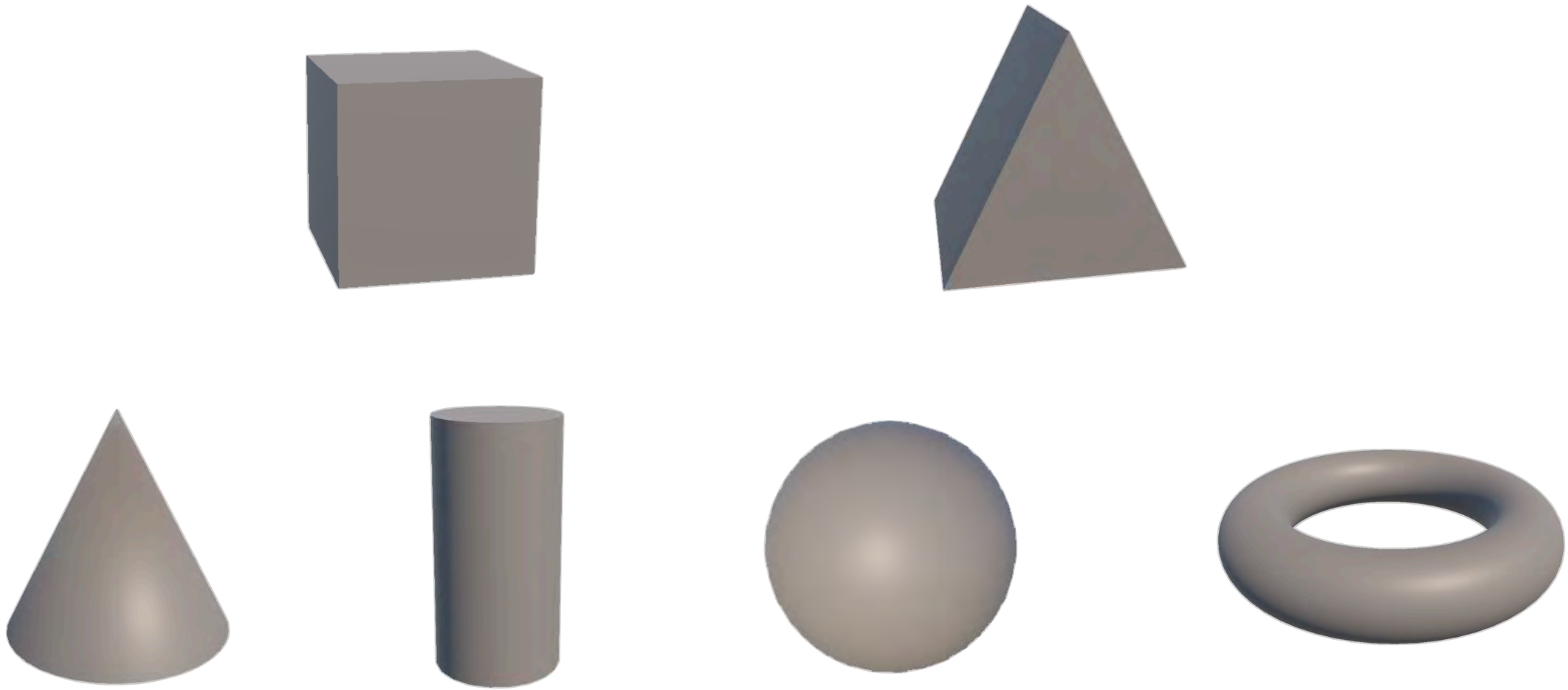
Zhu Wang
Assistant Professor of Computer Science



COMPUTER SCIENCE

Geometric Primitives

Geometric Primitives



Implicit Representation



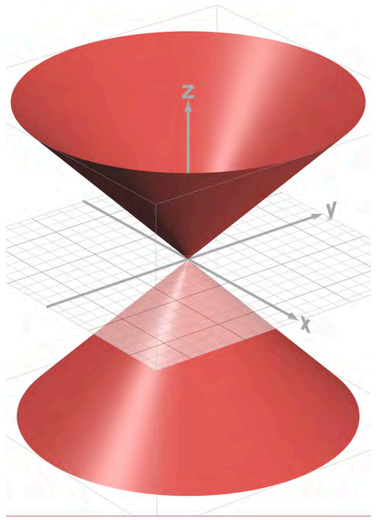
$$x^2 + y^2 + z^2 = r^2$$



$$(\sqrt{x^2 + y^2} - R)^2 + z^2 = r^2$$



$$x^2 + y^2 - z^2 = 0$$





$$x^2 + y^2 - z^2 = 0 \quad z \in [z_{min}, z_{max}]$$

Parametric Representation

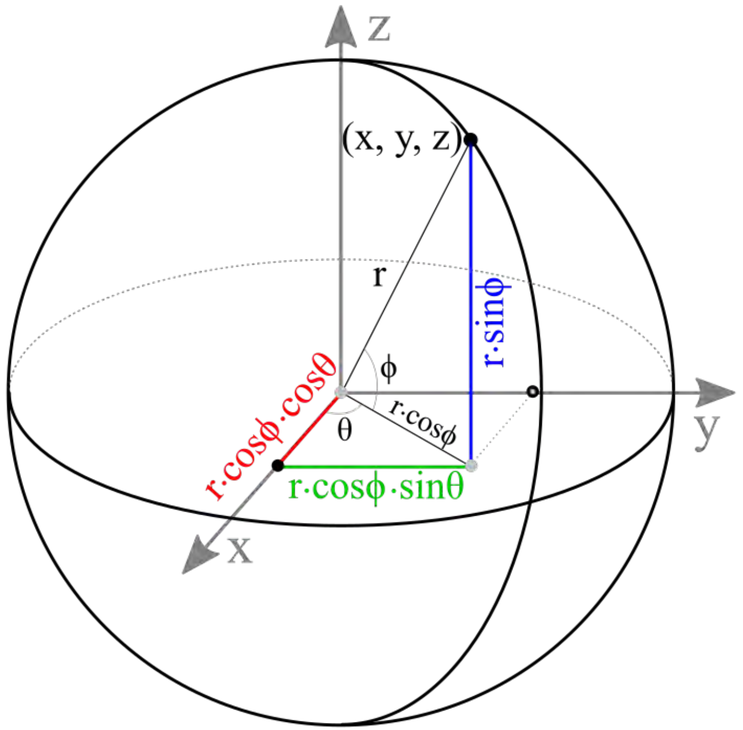
Implicit Representation $f(x, y, z) = 0$

- Inside/outside tests
- Intersections

Parametric Representation $(x, y, z) = f(u, v)$

- Given by function of parameters $f(u, v)$
- Generating points for sampling/drawing

Sphere



$$x^2 + y^2 + z^2 = r^2$$

[source: songho.ca]

For a circle,

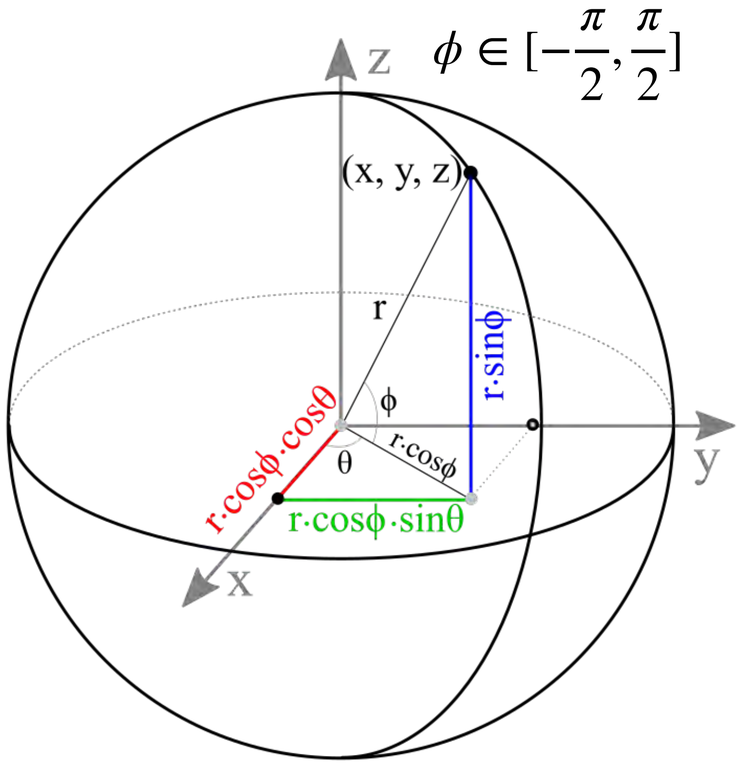
$$x(t) = r\cos t$$

$$y(t) = r\sin t$$

For a sphere, u - longitude, v-latitude

$$u \in [0,2\pi] \ , \ v \in [0,\pi]$$

Sphere



Longitude Angle Latitude Angle

$$u \in [0,2\pi] \ , \ v \in [0,\pi]$$

$$\rightarrow \quad x = r \sin v \cos u$$
$$y = r \sin v \sin u$$
$$z = r \cos v$$

Or

$$x = x_c + r \sin v \cos u$$
$$y = y_c + r \sin v \sin u$$
$$z = z_c + r \cos v$$

If centered at (x_c, y_c, z_c)

$$x^2 + y^2 + z^2 = r^2$$

[source: songho.ca]

Cylinder



$$x^2 + y^2 - z^2 = 0$$
$$z \in [z_{min}, z_{max}]$$

Longitude Latitude Height

$$u \in [0,2\pi] \ , \ v \in [0,1] \ , \ h$$

$$x = r \cos u$$

$$y = r \sin u$$

$$z = h \cdot v$$

Cone



$$x^2 + y^2 - z^2 = 0$$

Longitude Latitude Height

$$u \in [0,2\pi] \ , \ v \in [0,1] \ , \ h$$

$$x = r \cdot (1 - v) \cdot \cos u$$

$$y = r \cdot (1 - v) \cdot \sin u$$

$$z = h \cdot v$$

Torus/Donut



$$(\sqrt{x^2+y^2}-R)^2+z^2=r^2$$

Large circle Tube

$$u \in [0,2\pi] \ , \ v \in [0,2\pi]$$

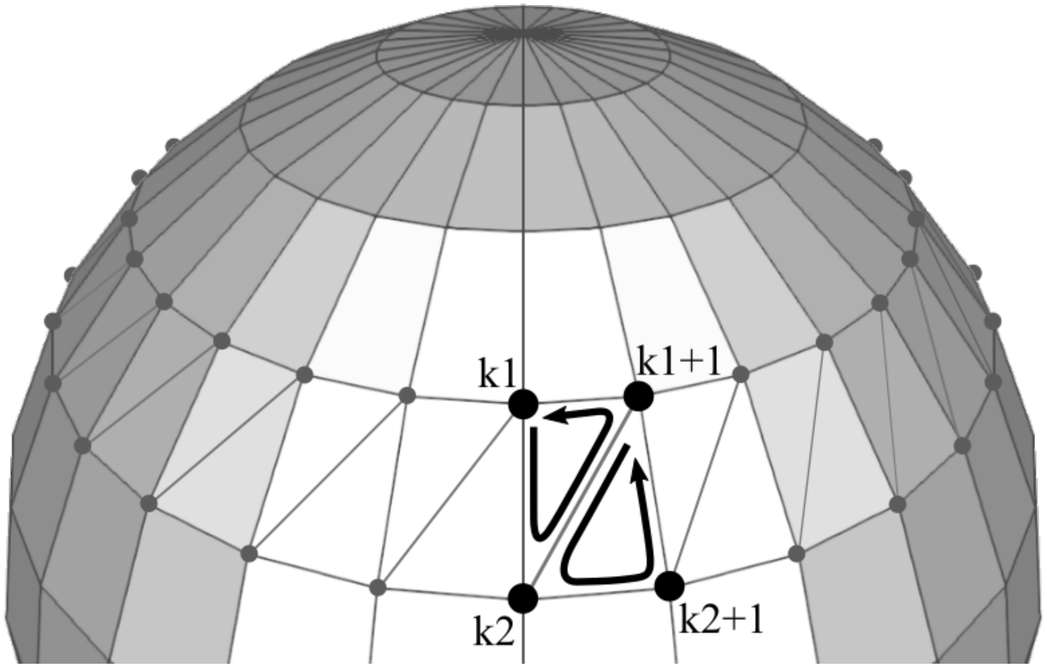
R Large circle radius r Tube radius

$$x = (R + r\cos v)\cos u$$

$$y = (R + r\cos v)\sin u$$

$$z = r\sin v$$

Generating a sphere in WebGL



[source: songho.ca]

```
for (let i = 0; i <= vSteps; i++) {
  const v = i * Math.PI / vSteps;
  const sinv = Math.sin(v);
  const cosv = Math.cos(v);

  for (let j = 0; j <= uSteps; j++) {
    const u = j * 2 * Math.PI / uSteps;
    const sinu = Math.sin(u);
    const cosu = Math.cos(u);
    const x = cosu * sinv;
    const y = cosv;
    const z = sinu * sinv;

    vertices.push(r * x, r * y, r * z);
  }
}

for (let i = 0; i < vSteps; i++) {
  for (let j = 0; j < uSteps; j++) {
    const k1 = (i * (uSteps + 1)) + j;
    const k2 = first + uSteps + 1;
    indices.push(k1, k2, k1 + 1);
    indices.push(k2, k2 + 1, k1 + 1);
  }
}
```

WebGL

WebGL

Initialization

```
<canvas id="canvas" width="800" height="600"></canvas>
<script>window.onload = function() {
    const canvas = document.getElementById('canvas');
    const gl = canvas.getContext('webgl2');
    if (!gl)
        { alert("WebGL not supported"); }
    else
        { console.log("WebGL initialized");
```

init()

```
    gl.enable(gl.DEPTH_TEST);
    gl.clearColor(0, 0, 0, 1);
    gl.clear(gl.COLOR_BUFFER_BIT | gl.DEPTH_BUFFER_BIT);
```

drawScene()

```
} </script>
```

Scene Data

```
Vertices: [
    -1, -1, -1,   // 0
     1, -1, -1,   // 1
     1,  1, -1,   // 2
    -1,  1, -1,   // 3
    -1, -1,  1,   // 4
     1, -1,  1,   // 5
     1,  1,  1,   // 6
    -1,  1,  1    // 7
  ]
```

```
Indices: [
    // front face
    0, 1, 2,  0, 2, 3,
    // back face
    4, 5, 6,  4, 6, 7,
    // left face
    0, 4, 7,  0, 7, 3,
    // right face
    1, 5, 6,  1, 6, 2,
    // top face
    3, 2, 6,  3, 6, 7,
    // bottom face
    0, 1, 5,  0, 5, 4
  ]
```

```
Colors: [
  // Vertex 0: (-1, -1, -1)
  1.0, 0.0, 0.0, 1.0,    // Red
  // Vertex 1: ( 1, -1, -1)
  0.0, 1.0, 0.0, 1.0,    // Green
  // Vertex 2: ( 1,  1, -1)
  0.0, 0.0, 1.0, 1.0,    // Blue
  // Vertex 3: (-1,  1, -1)
  1.0, 1.0, 0.0, 1.0,    // Yellow
  // Vertex 4: (-1, -1,  1)
  1.0, 0.0, 1.0, 1.0,    // Magenta
  // Vertex 5: ( 1, -1,  1)
  0.0, 1.0, 1.0, 1.0,    // Cyan
  // Vertex 6: ( 1,  1,  1)
  1.0, 1.0, 1.0, 1.0,    // White
  // Vertex 7: (-1,  1,  1)
  0.0, 0.0, 0.0, 1.0     // Black
];
```

Buffer Binding

Binding scene data to GPU

```
const positionBuffer = gl.createBuffer();

gl.bindBuffer(gl.ARRAY_BUFFER, positionBuffer);
gl.bufferData(gl.ARRAY_BUFFER,
    new Float32Array(data.vertices), gl.STATIC_DRAW);

const indexBuffer = gl.createBuffer();
gl.bindBuffer(gl.ELEMENT_ARRAY_BUFFER, indexBuffer);
gl.bufferData(gl.ELEMENT_ARRAY_BUFFER,
    new Uint16Array(data.indices), gl.STATIC_DRAW);
```

Vertex Shader

```
#version 300 es
in vec3 aPosition;    // vertex position
in vec3 aColor;       // vertex color
uniform mat4 uModelView;    // model transformation
uniform mat4 uProjection;   // projection transformation
out vec3 vColor;      // vertex color

void main() {
   // Apply model and projection transformations
   gl_Position = uProjection * uModelView
                     * vec4(aPosition, 1.0);
   // Pass color data to the fragment shader
   vColor = aColor;
}
```

Fragment Shader

```
#version 300 es
precision mediump float;

in vec3 vColor;         // interpolated from vs
out vec4 fragColor;     // the final pixel color

void main() {
    // Pass color data to the fragment shader
    fragColor = vec4(vColor, 1.0);
}
```

Compiling GLSL




```
function createShader(gl, type, source) {
    let shader = gl.createShader(type);
    gl.shaderSource(shader, source);
    gl.compileShader(shader);
    if (!gl.getShaderParameter(shader,
                               gl.COMPILE_STATUS)) {
        throw new Error(gl.getShaderInfoLog(shader));
      }
      return shader;
    }
```

Fragment Shader

```
#version 300 es
precision mediump float;

in vec3 vColor;         // interpolated from vs
out vec4 fragColor;     // the final pixel color

void main() {
    // Pass color data to the fragment shader
    fragColor = vec4(vColor, 1.0);
}
```

Linking and Creating Program

```
    let vs = createShader(gl, gl.VERTEX_SHADER, vsSource);
    let fs = createShader(gl, gl.FRAGMENT_SHADER, fsSource);
    let prog = gl.createProgram();
    gl.attachShader(prog, vs);
    gl.attachShader(prog, fs);
    gl.linkProgram(prog);
    if (!gl.getProgramParameter(prog, gl.LINK_STATUS)) {
        throw new Error(gl.getProgramInfoLog(prog));
```

Uniform Variables

Global parameters

```
#version 300 es
in vec3 aPosition;
in vec3 aColor;

uniform float uTime;
out vec3 vColor;

void main() {
    gl_Position = vec4(aPosition, 1.0);
    vColor = aColor;
}
```

Attribute/Uniform locations

```
const timeLoc = gl.getUniformLocation(program, "uTime");

//current time in ms
let deltaTime = Date.now();

gl.useProgram(program);

//set time in seconds
gl.uniform1f(timeLoc, deltaTime/1000.0);
```

Formats of Uniform Variables

```
gl.uniform3f(location, x, y, z);
gl.uniform1i(location, x);
gl.uniform3fv(location, v);
gl.uniformMatrix4fv(location, isTranspose, mat4);
```

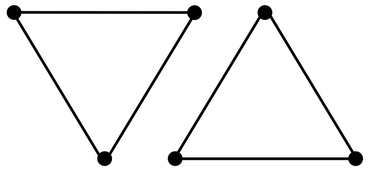
$$M = \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \\ m_{30} & m_{31} & m_{32} & m_{33} \end{bmatrix}$$

```
mat4 =
[m00, m10, m20, m30,//1st column
 m01, m11, m21, m31,//2nd column
 m02, m12, m22, m32,//3rd column
 m03, m13, m23, m33];//4th column
```

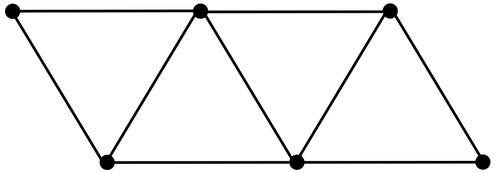
Rendering

```
gl.drawArrays(mode, first, count);
```

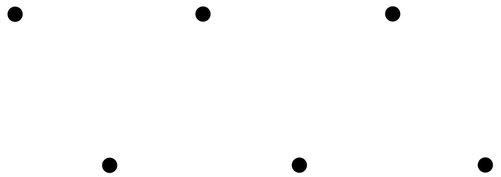
GL_TRIANGLES



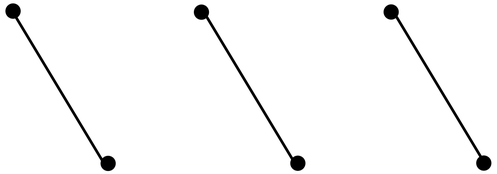
GL_TRIGANGLE_STRIP



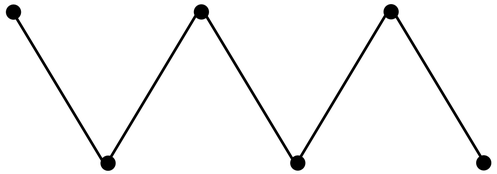
GL_POINTS



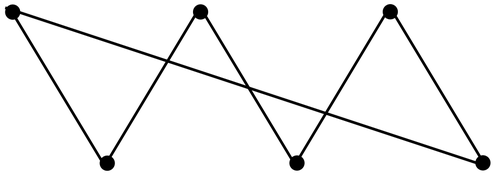
GL_LINES



GL_LINE_STRIP



GL_LINE_LOOP



GL_TRIGANGLE_FAN

